MC Exchange (Domilea) - v1

Product Overview

Field	Value
🎯 Target date	TBD
🟡 Document status	DRAFT
📥 Team members	@Zonayad Akbar

Quick Links

Field	Value
🎨 Designs	TBD
📽️ Loom demo	
🗂️ Work tracker	TBD

🎯 Objective

MC Exchange (marketed as Domilea) is a trust-based marketplace for buying, selling, and verifying motor carrier (MC) authorities. Sellers list their MC authority with safety history, fleet data, insurance filings, and compliance documents. Buyers browse verified listings, unlock them with credits, make offers, and transact through an admin-mediated escrow. Every listing carries a trust score (high 80+ green, medium 50–79 yellow, low <50 red) derived from FMCSA data, CreditSafe reports, seller verification, and platform activity. Three user roles operate the system — buyers, sellers, and admins — and monetization runs on three parallel paths: subscription plans with monthly credits, one-time credit packs, and CarrierPulse — a separate subscription product that bundles carrier intelligence tools (CreditSafe reports, Chameleon Check, Safety Improvement Report, Eva AI) outside the marketplace credit system. A full-service admin layer moderates every listing, mediates every offer, verifies every document, and manually releases escrow payouts through Stripe Connect.

📊 Success Metrics

North Star Goal	North Star Metric
Maximize the volume of trusted MC authority transactions closed through the platform	Transactions moved from offer-accepted to escrow-released per month

Goal	Metric
Marketplace & Discovery: Give buyers a fast, trustworthy way to find and evaluate MC authorities without leaving the platform.	1. Avg time from marketplace entry → listing unlock: target < 3 min 2. % of listings unlocked at least once within 30 days of publish: target > 60% 3. Masked-to-unlocked conversion rate per listing: target > 15% 4. % of buyers who run FMCSA/CreditSafe lookups before unlock: target > 70%
Trust & Verification: Make trust the reason buyers pick Domilea over cold outreach.	1. % of listings with all 4 verification layers complete (FMCSA, CreditSafe, documents, seller identity): target > 80% 2. Avg buyer time from signup → Stripe Identity verified: target < 10 min 3. % of sellers `sellerVerified` at time of first listing publish: target > 75% 4. % of admin-run due diligence reports flagged for further review: tracked (no target)
Offers & Transactions: Close deals end-to-end inside the platform with minimal buyer/seller drop-off.	1. Avg time from offer submitted → admin-forwarded to seller: target < 24 hrs 2. % of forwarded offers that convert to a transaction: target > 35% 3. Avg time from deposit paid → escrow confirmed: target < 48 hrs 4. % of transactions that complete without dispute: target > 90%
Subscriptions & Credits: Turn browsing users into paying subscribers and retain them past the first billing cycle.	1. Free-to-paid conversion rate: target > 8% 2. Month-2 subscription retention: target > 65% 3. % of credits granted that are consumed in the billing period: target > 60% 4. Involuntary churn rate from failed payments: target < 5%
CarrierPulse (Standalone Product): Operate CarrierPulse as its own revenue line independent of marketplace activity.	1. Standalone CarrierPulse subscribers (no active marketplace subscription): tracked 2. Avg CreditSafe reports run per active CarrierPulse subscriber: tracked 3. Chameleon Check / Safety Report usage per active subscriber: tracked
Messaging & Realtime: Keep both sides informed in real time so offers and transactions don't stall.	1. Avg time from message sent → read: target < 2 hrs during business hours 2. % of transactions with at least one message thread: target > 80% 3. Notification delivery success rate (socket + nav badge): target > 99%

Admin & Moderation: Keep admin throughput high without becoming a bottleneck to deal flow.	1. Avg admin review time per listing: target < 4 business hrs 2. Avg admin review time per offer: target < 24 business hrs 3. % of escrow releases processed via instant payout: target > 40% 4. Admin actions per active closing: target < 6
AI Assist: Make AI the fastest path to answers during listing review and buyer inquiries.	1. % of admin listing reviews that run AI due diligence: target > 50% 2. Avg AI Chat thread depth (messages per thread): target > 3 3. User satisfaction with AI responses: target > 7/10

🤔 Assumptions

User & Customer Assumptions

- Sellers are existing motor carrier authority owners who want to sell their MC to another operator, not speculative investors.
- Buyers are operators actively shopping for an MC authority and can produce a deposit within days of an accepted offer.
- Users accept that every offer is reviewed by an admin before it reaches the seller — admin mediation is part of the trust positioning, not a temporary workaround.
- Buyers will complete Stripe Identity verification before unlocking a listing or submitting an offer.
- Sellers will complete Stripe Connect onboarding before payout is possible.
- Both sides accept that credential transfer (Highway/FMCSA/IRP/insurance portal logins) happens inside the platform's encrypted credential vault, not over email.

Data & AI Assumptions

- FMCSA public data is sufficiently accurate and current for lookup by MC or DOT, with a 24-hour cache acceptable for reports.
- CreditSafe business reports are reliable enough to surface to buyers and admins for due diligence; subscription tiers gate who sees them and at what depth.
- OpenAI Assistants can synthesize FMCSA + CreditSafe data into a useful due diligence narrative for admin reviewers — not a substitute for admin judgment.
- Trust score is a composite heuristic, not an audit. A green score does not guarantee clean authority; a red score does not always mean fraud.

Technical & Integration Assumptions

- Stripe is the only payment rail. Subscriptions, deposits, final payments, credit packs, consultations, CarrierPulse, instant payouts, and Stripe Identity all route through Stripe.

- Stripe Connect (Express accounts) is the only seller payout mechanism.
- Two separate Stripe webhook endpoints exist: `/api/webhooks/stripe` (marketplace, subscriptions, deposits, payouts) and `/api/consultations/webhook` (consultation payments only). Both must remain before `express.json()` middleware to preserve raw body for signature verification.
- AWS S3 is specified for document storage but the runtime implementation is local disk via Multer; presigned URLs are not yet in production.
- Redis is assumed available in all production environments (rate limiting, FMCSA/carrier report cache). Absence silently degrades behavior.
- socket.io is the only realtime channel. Clients connect after login with the same JWT used for REST auth.
- Frontend is a React SPA on Vercel (`https://www.domilea.com`); backend is Express on Heroku (`mcxchange` app) with JawsDB MySQL.

Workflow Assumptions

- Dispatchers, brokers, and other trucking-adjacent roles are not first-class users — only buyer/seller/admin.
- Every listing goes through admin review before publish. No auto-approve path exists.
- Every offer goes through admin review before reaching the seller. Sellers do not respond to raw buyer offers.
- Every transaction requires all three parties (buyer, seller, admin) to sign off before escrow releases.
- Document verification is manual; admin marks documents VERIFIED or REJECTED.
- Credential release (MC transfer login handoff) is a manual action by the seller, followed by manual revocation capability if needed.

Subscription & Billing Assumptions

- Stripe is the authoritative source of subscription lifecycle. The local `subscriptions` table and `users` credit fields mirror Stripe; discrepancies are reconciled via webhooks.
- Users may hold two parallel Stripe subscriptions: one marketplace subscription (STARTER/PREMIUM/ENTERPRISE/VIP_ACCESS) and one CarrierPulse subscription. Only one of each is supported at a time.
- Credit grants happen on initial checkout and on each monthly renewal via `invoice.paid` with `billing_reason === 'subscription_cycle'`.
- Credits are consumed when a buyer unlocks a listing; unlocked listings are permanent (no re-debit for re-access).
- Credit packs are one-off purchases that stack on top of subscription credits.
- Identity verification via Stripe Identity is required before a buyer can unlock, message, or make an offer.
- Listing fees ($49.99) are one-time payments per listing, not per seller, and may be toggled on/off per platform setting (`listingPaymentRequired`).

Compliance & Document Assumptions

- Sellers upload authority-related compliance documents; admin reviews and marks each VERIFIED or REJECTED.
- Documents are attached to either a listing or a transaction, never floating.
- Transaction credentials (encrypted Highway/FMCSA/IRP logins) are the primary artifact of authority transfer; they are stored AES-encrypted with per-credential IV/authTag.
- Users accept terms of service per listing at transaction time, not once globally. Terms acceptances are versioned and PDF-archived.
- Blocked users may submit a dispute via a public token-protected form; admins resolve or reject; auto-unblock can be triggered after a defined window.

🌟 Milestones

Coming Soon

📋 Feature & Requirements

Marketplace & Discovery

Description: The public-facing surface where buyers search, filter, and preview MC listings before subscribing or unlocking. Discovery is intentionally partial — MC and DOT numbers are masked until the buyer unlocks, creating the pressure to subscribe.

Narrative: Discovery is the top of the funnel. A buyer lands on the marketplace, filters by state, fleet size, safety rating, and authority age, and sees enough to evaluate fit but not enough to bypass the platform. FMCSA lookups are free and public — the value add is the curation, trust score, and seller verification. The VIP Marketplace is a walled section visible only to Enterprise and VIP subscribers.

Requirement	User Story	Importance	Jira Issue	Notes
Public marketplace browse	As a visitor, I browse MC listings without signing up.	HIGH		GET /api/listings, Listing model, public

Listing detail view with masked identifiers	As a visitor, I see listing details with MC/DOT masked until unlock.	HIGH	GET /api/listings/:id, MCDetailPageV2
FMCSA lookup by MC or DOT	As a visitor, I look up any carrier's FMCSA data from the landing page.	HIGH	fmcsaRoutes (8 endpoints), 24h Redis cache
Aggregated carrier report	As a visitor, I see a combined FMCSA snapshot for any DOT.	HIGH	GET /api/carrier-data/report/:dotNumber
Search filters	As a buyer, I filter listings by state, fleet size, authority age, price, trust tier.	HIGH	listingRoutes query params
VIP Marketplace	As an Enterprise/VIP subscriber, I browse premium listings gated from other tiers.	HIGH	GET /api/listings/vip, requireEnterpriseSubscription
Saved listings	As a buyer, I save listings for later review.	MEDIUM	SavedListing model — **backend-only; no UI wired**
Unlocked listings list	As a buyer, I see every listing I've unlocked.	HIGH	GET /api/listings/unlocked, UnlockedListing model
Premium request	As a buyer, I request premium unlock for a specific listing.	MEDIUM	POST /api/buyer/premium-requests, PremiumRequest model

---

Listing Management

Description: The seller's workspace for creating, editing, submitting, and maintaining MC listings, plus the admin review and moderation surface that gates every listing before publish.

Narrative: Sellers create a listing via a 3-step wizard that auto-fills FMCSA data from an MC or DOT number, then adds fleet, insurance, asking price, and documents. Optional $49.99 listing

fee (togglable per platform setting). Every listing enters PENDING_REVIEW and cannot reach buyers until an admin approves it and optionally sets `listingPrice` or `freeToUnlock`. This admin gate is the product's trust lever — it is the reason buyers pay to unlock.

Requirement	User Story	Importance	Jira Issue	Notes
Seller welcome / onboarding	As a new seller, I pre-fill my profile from my MC number.	HIGH		SellerWelcomePage, fmcsaLookupByMC → updateProfile
Create listing	As a seller, I create a listing with FMCSA auto-fill, fleet, insurance, and pricing.	HIGH		POST /api/listings, SellerCreateListingPage
Pay listing fee	As a seller, I pay the $49.99 listing fee via Stripe Checkout.	HIGH		POST /api/seller/listing-fee/checkout, toggleable via platform setting
Submit for review	As a seller, I submit a DRAFT listing for admin review.	HIGH		Lifecycle: DRAFT → PENDING_REVIEW → ACTIVE/REJECTED
Edit and delete listings	As a seller, I edit or delete my listings from the dashboard.	HIGH		PUT/DELETE /api/listings/:id
Seller dashboard overview	As a seller, I see my listings, offers, and messages on one page.	HIGH		GET /api/seller/dashboard
Admin pending queue	As an admin, I see every pending listing in one queue.	HIGH		GET /api/admin/listings/pending, AdminPendingReviewPage

Admin approve/reject with notes	As an admin, I approve a listing (optionally override price and free-to-unlock) or reject with a reason.	HIGH	POST /api/admin/listings/:id/approve or /reject, AdminAction audit row
Admin listing edit	As an admin, I edit any field on any listing.	HIGH	PUT /api/admin/listings/:id, AdminAllListingsPage
Admin-created listings	As an admin, I create a listing on behalf of a seller (and optionally create the user too).	HIGH	POST /api/admin/listings, POST /api/admin/users/with-listing

---

Trust & Verification

Description: The layered verification system that produces a trust score and makes buyers confident enough to transact. Covers buyer identity (Stripe Identity), seller identity (email + manual admin verification), listing-level data (FMCSA, CreditSafe, documents), and cross-reference due diligence.

Narrative: Trust is the product's real differentiator. Four independent verification surfaces feed a composite trust score: buyer Stripe Identity, seller admin-verified status, per-listing document review, and external data (FMCSA safety/authority/insurance history, CreditSafe business reports). Admins run AI-assisted due diligence against FMCSA + CreditSafe data before approving any listing. The dispute system handles false-positive blocks when a user claims they were blocked in error.

Requirement	User Story	Importance	Jira Issue	Notes
Email verification	As a new user, I verify my email before accessing most features.	HIGH		EmailVerificationToken, POST /api/auth/verify-email

Stripe Identity verification	As a buyer, I complete Stripe Identity to unlock listings, message, and offer.	HIGH	identityRoutes, stripeVerificationSessionId on User, VerificationRequiredRoute gate
Seller manual verification	As an admin, I mark a seller verified after reviewing their documents.	HIGH	POST /api/admin/users/:id/verify-seller, sellerVerified flag, trust +20
Document upload and review	As a seller, I upload compliance documents; as an admin, I mark them VERIFIED/REJECTED.	HIGH	documentRoutes, Document model, status enum
FMCSA data integration	As any user, I view live FMCSA safety, authority, insurance, and cargo data for a DOT.	HIGH	fmcsaRoutes, 24h Redis cache
CreditSafe admin console	As an admin, I run CreditSafe searches and reports to qualify buyers and sellers.	HIGH	creditsafeRoutes (6 endpoints), admin-only
Buyer CreditSafe access	As a Professional+ buyer, I run CreditSafe reports on listings I'm evaluating.	HIGH	/api/buyer/creditsafe/*, gated by subscription tier
One-off credit report purchase	As a buyer without a subscription, I buy a single $55 CreditSafe report.	MEDIUM	POST /api/buyer/creditsafe/checkout, creditsafePurchasedReport
AI due diligence	As an admin, I run OpenAI-synthesized due diligence against FMCSA + CreditSafe data before approving.	HIGH	POST /api/admin/due-diligence/analyze/:mcNumber, AdminAIDueDiligence page
Trust score display	As any user, I see a listing's trust tier (green/yellow/red).	HIGH	trustScore field on User; derived tier on display

Terms of service acceptance per listing	As a buyer, I accept terms before submitting an offer on a specific listing.	HIGH	UserTermsAcceptance model, versioned, PDF-archived
Account dispute (blocked user recovery)	As a blocked user, I submit a dispute form to request review.	MEDIUM	disputeRoutes, AccountDispute model, token-protected public page
Reviews display	As a visitor, I see review history on a user profile.	LOW	GET /api/users/:id/reviews exists — **no create-review flow in product today**

Offers & Transactions

Description: The multi-step, admin-mediated deal flow from offer submission through escrow release. Includes offers, transactions, documents, transaction-scoped messaging, and the encrypted credential vault for MC authority handoff.

Narrative: Every offer is reviewed by an admin before it reaches the seller. This is the product's trust mechanism, not a workaround. Once forwarded, the seller accepts, rejects, or counters. On acceptance, a Transaction is created and the buyer pays a deposit via Stripe Checkout. Both parties accept transaction-level terms, upload documents, and progress the status through a shared TransactionRoom. Admin confirms escrow, the seller releases encrypted login credentials for authority transfer portals, and admin triggers payout via Stripe Connect (standard or instant). The TransactionRoom is a shared route — buyer, seller, and admin see the same page with role-specific actions.

Requirement	User Story	Importance	Jira Issue	Notes
Create offer	As a verified buyer, I submit an offer with amount, message, and buy-now flag.	HIGH		POST /api/offers, Offer model, initial status PENDING_ADMIN
Admin offer moderation	As an admin, I forward, approve, reject, or delete a buyer's offer.	HIGH		/api/admin/offers/*, adminReviewedBy/At, adminMessageToSeller

Seller offer actions (FLAG)	As a seller, I accept, reject, or counter a forwarded offer.	HIGH	**Backend ready (POST /offers/:id/accept, /reject, /counter); no UI controls yet. Flagged for product review — decide whether seller acts directly or admin continues to mediate.**
Buyer offer management	As a buyer, I view my offers, accept a counter, or withdraw an offer.	HIGH	/api/offers/my-offers, /accept-counter, /withdraw
Deposit payment	As a buyer, I pay the deposit via Stripe Checkout to open a transaction.	HIGH	POST /api/offers/:id/deposit-checkout → Transaction created
Transaction Room (shared workspace)	As a buyer/seller/admin, I work the transaction end-to-end from one page.	HIGH	TransactionRoomPage, /transaction/:id, role-specific actions
Three-party approval	As a buyer, seller, and admin, each of us approves the transaction before escrow releases.	HIGH	POST /transactions/:id/{buyer,seller,admin}/approve
Transaction documents	As any participant, I upload and view transaction-level documents (BOS, BOL, authority paperwork).	HIGH	documentRoutes with transactionId, admin verification required
Transaction messaging	As a participant, I message within a transaction with role visible on each message.	HIGH	TransactionMessage model, POST /transactions/:id/messages
Transaction timeline	As any participant, I see a timestamped audit trail of every transaction event.	MEDIUM	TransactionTimeline model, populated by service layer

Final payment	As a buyer, I pay the remaining balance via Stripe Checkout.	HIGH	POST /transactions/:id/final-payment-checkout
Admin verify payments	As an admin, I manually verify a deposit or final payment.	HIGH	POST /transactions/:id/admin/verify-{deposit,payment}/:paymentId
Admin confirm escrow	As an admin, I confirm escrow is satisfied before any payout can release.	HIGH	POST /transactions/:id/admin/confirm-escrow
Encrypted credential vault	As a seller, I store Highway/FMCSA/IRP/insurance logins inside the platform (AES-encrypted) and release them to the buyer at handoff.	HIGH	credentialRoutes, TransactionCredential model, encryptedUsername/encryptedPassword
Credential release + revoke	As a seller, I release credentials; as an admin, I can revoke the release if the deal reverses.	HIGH	POST /credentials/transaction/:txId/release and /revoke
Payout release	As an admin, I release payout to the seller's Stripe Connect account (standard or instant debit-card).	HIGH	POST /api/admin/transactions/:id/release-payout, checkInstantPayoutEligibility
Admin-created transactions	As an admin, I create a transaction manually between an available buyer and listing (off-platform origin).	HIGH	POST /transactions/admin/create, AdminActiveClosingsPage

---

Subscriptions & Credits

Description: The primary monetization system for marketplace access. Four subscription tiers, three one-time credit packs, and a strict three-layer sync rule between Stripe, the `subscriptions` table, and the `users` credit fields. Credits are consumed on listing unlock.

Narrative: This pillar is load-bearing. Subscription state lives in three places — Stripe (authoritative), the `subscriptions` table (local mirror), and the `users` table (credit balance

and Stripe customer ID). Every subscription change must keep all three in sync. The Stripe webhook at `/api/webhooks/stripe` is the primary reconciliation path. Credits are debited only from `users.totalCredits` and `users.usedCredits`; the `Subscription.creditsRemaining` column is maintained for reporting but is not the source of truth at debit time.

Subscription plans (canonical values from `src/services/pricingConfigService.ts`, overridable via admin PlatformSetting):

Plan	Price/mo	Price/yr	Credits/mo	Gating
STARTER	$19.99	$192.00	4	Basic marketplace access
PREMIUM	$39.99	$383.99	10	Auto-unlock premium listings, EVA AI access
ENTERPRISE	$79.99	$767.99	15	VIP Marketplace access, free credit reports
VIP_ACCESS	$399.99	$3,839.99	999	Unlimited unlocks, $399 credit toward an MC purchase

Credit packs (one-time, stack on top of subscription): `pack_5` $24.99 / 5 credits, `pack_10` $44.99 / 10 credits, `pack_25` $99.99 / 25 credits.

Deprecated / renamed plans (handled in tech debt section): `PROFESSIONAL` was migrated to PREMIUM in production but the enum and middleware (`requireProfessionalSubscription`) still reference it. `PACKAGE_TOOL` ($14.99) is the Pulse Bundle subscription — documented under the CarrierPulse pillar, not here.

Requirement	User Story	Importance	Jira Issue	Notes
Public pricing page	As a visitor, I see all subscription plans and credit packs.	HIGH		GET /api/credits/plans, GET /api/credits/packs, PricingPage
Subscription checkout	As a buyer, I subscribe via Stripe Checkout.	HIGH		POST /api/buyer/subscription/checkout, requires identity verified

Subscription verify (post-checkout sync)	As a buyer returning from Stripe, the system reconciles my subscription state.	HIGH	POST /api/buyer/subscription/verify
Cancel subscription (self-serve)	As a subscriber, I cancel my plan from the buyer subscription page.	HIGH	POST /api/buyer/subscription/cancel
Admin cancel subscription	As an admin, I cancel any user's subscription.	HIGH	POST /api/admin/users/:id/cancel-subscription
Buy credit pack	As a buyer, I purchase one-time credit packs on top of my subscription.	HIGH	POST /api/credits/packs/:packId/checkout
Admin adjust credits	As an admin, I grant, debit, or refund credits on any user account.	HIGH	POST /api/admin/users/:id/credits, POST /api/credits/bonus, /refund
Unlock listing (primary credit debit)	As a buyer, I spend 1 credit to unlock a listing's MC/DOT and full detail.	HIGH	POST /api/listings/:id/unlock, UnlockedListing, CreditTransaction(USAGE)
Free-to-unlock listings	As a buyer with an active subscription, I unlock admin-flagged free listings without spending a credit.	HIGH	listing.freeToUnlock; creditsUsed=0

Stripe webhook reconciliation	As the system, every Stripe event mutates the DB exactly once.	HIGH	POST /api/webhooks/stripe, ProcessedWebhookEvent idempotency
Monthly credit grant on renewal	As a subscriber, my credits reset at each billing cycle.	HIGH	invoice.paid with billing_reason === 'subscription_cycle'
Credit balance + history	As a buyer, I see my balance and every credit transaction.	MEDIUM	Balance visible in UI; GET /api/credits/history — **backend ready; no frontend caller**
Admin pricing configuration	As an admin, I change plan prices, credit counts, and listing fees without a deploy.	HIGH	GET/PUT /api/admin/pricing, PlatformSetting

---

CarrierPulse (Standalone Product)

Description: A separate subscription product layered on top of the same platform — CarrierPulse bundles carrier intelligence tools (CreditSafe reports, Chameleon Check, Safety Improvement Report, Eva AI, bulk FMCSA lookups) and is sold independently of marketplace subscriptions.

Narrative: CarrierPulse runs on its own Stripe subscription (`carrierPulseStripeSubId` on the User record), uses its own access flag (`carrierPulseAccess`), and has its own checkout flow. A user can hold a CarrierPulse subscription with zero marketplace activity, or stack it on top of a marketplace subscription. The Pulse Bundle plan ($14.99/mo, 0 marketplace credits) is the entry-level CarrierPulse subscription. Preview pages exist for all CarrierPulse surfaces even for non-subscribers — they are marketing conversion pages.

Requirement	User Story	Importance	Jira Issue	Notes

Pulse Bundle subscription	As a non-marketplace user, I subscribe to the $14.99 Pulse Bundle to access carrier tools only.	HIGH	PACKAGE_TOOL plan, PulseBundlePage, POST /api/buyer/subscription/checkout
CarrierPulse checkout	As any user, I subscribe to CarrierPulse as a standalone product.	HIGH	POST /api/buyer/carrier-pulse/checkout, createCarrierPulseCheckout
CarrierPulse access check	As the system, I gate CarrierPulse features on `carrierPulseAccess` flag.	HIGH	GET /api/buyer/carrier-pulse/access
CreditSafe company search	As a CarrierPulse subscriber, I search any company by name.	HIGH	carrierPulseCreditsafeSearch, CarrierPulsePage
CreditSafe report view	As a CarrierPulse subscriber, I view full CreditSafe reports on a company.	HIGH	carrierPulseCreditsafeReport
Chameleon Check	As a CarrierPulse subscriber, I run Chameleon-carrier detection by DOT.	HIGH	ChameleonCheckPage, carrier-pulse access gate
Safety Improvement Report	As an admin (or subscriber), I generate a Safety Improvement Report for a carrier.	MEDIUM	SafetyImprovementReportPage, fmcsaGetSMSData + getCarrierReport
Eva AI assistant	As a subscriber, I use the Eva AI chat assistant for carrier research questions.	MEDIUM	EvaAIPreviewPage, aiChatRoutes — preview-level today, not a full assistant surface
Standalone credit report purchase	As a non-subscriber, I buy a one-off $55 credit report.	MEDIUM	POST /api/buyer/creditsafe/checkout, CreditReportPurchasePage

---

Messaging & Notifications

Description: All in-platform user-to-user and user-to-admin communication, plus the realtime notification layer that keeps offers, transactions, and messages alive.

Narrative: Messages are platform-scoped — buyers and sellers cannot exchange contact information directly until a transaction is opened. Admin inquiries are a separate message track (the inquiry inbox), used for listing questions that haven't reached offer stage. Notifications drive the nav-bar badge counts and socket.io events; full notification center UI is not yet shipped.

Requirement	User Story	Importance	Jira Issue	Notes
User-to-user DMs	As a buyer or seller, I message the other party through the platform.	HIGH		messageRoutes, Message model, role-gated
Conversation list	As a user, I see all my conversations with unread counts.	HIGH		GET /api/messages/conversations
Single conversation view	As a user, I read and reply to	HIGH		GET /api/messages/conversations/:partnerId

	one conversation.		
Mark conversation as read	As a user, my reads sync across devices.	HIGH	PUT /api/messages/conversations/:partnerId/read
Admin inquiries	As a buyer, I send an inquiry to admin about a specific listing.	HIGH	POST /api/messages/inquiries, sendInquiryToAdmin
Admin inquiry inbox	As an admin, I reply to buyer inquiries from a single inbox.	HIGH	AdminMessagesPage
Realtime updates	As a user, I receive new-message, new-offer, and transaction-update events in real time.	HIGH	socket.io, JWT-authenticated, src/websocket/index.ts

Nav badge counts	As a user, I see unread messages and notifications on the nav.	HIGH	GET /api/notifications/nav-counts
Notification records	As the system, every significant event produces a Notification row.	HIGH	Notification model, types: OFFER/MESSAGE/VERIFICATION/REVIEW/TRANSACTION/SYSTEM/PAYMENT

Admin & Moderation

Description: The admin operations surface — moderation queues, user management, financial tooling, platform configuration, and the Active Closings workspace where admins manage deals in flight.

Narrative: Admin is not an afterthought role here — it is a primary operator of the marketplace. Admins moderate every listing, mediate every offer, verify every document, approve every escrow release, and run financial tooling (invoices, Stripe balance, payouts). The Active Closings page is the admin war room — available buyers, available listings, and manual transaction creation all live here.

Requirement	User Story	Importance	Jira Issue	Notes
Admin dashboard	As an admin, I see KPIs and the pending listing queue on login.	HIGH		GET /api/admin/dashboard, AdminDashboard

User management	As an admin, I view, edit, verify, block, unblock, role-change, and adjust credits on any user.	HIGH	/api/admin/users/*, AdminUsersPage
Activity log	As an admin, I search every AdminAction across the platform.	HIGH	AdminAction model, GET /api/admin/activity-log
Listing moderation	As an admin, I approve, reject, or edit any listing.	HIGH	AdminPendingReviewPage, AdminAllListingsPage
Offer moderation	As an admin, I forward, approve, reject, or delete any offer.	HIGH	AdminOffersPage
Active Closings workspace	As an admin, I create and manage in-flight transactions manually.	HIGH	AdminActiveClosingsPage, POST /transactions/admin/create
Invoice generator (Stripe-backed)	As an admin, I create, send, and void Stripe invoices for a user.	MEDIUM	AdminInvoiceGenerator — **invoices live in Stripe only, no local mirror table**
Stripe balance + transactions	As an admin, I see the Stripe balance, transactions, and balance transactions.	HIGH	/api/admin/stripe/*
Instant payout eligibility	As an admin, I check and trigger instant debit-card payouts to a seller.	HIGH	checkInstantPayoutEligibility, adminReleasePayout

Consultation management	As an admin, I manage paid consultations, update status, and refund.	MEDIUM	consultationRoutes (admin half), AdminConsultationsPage
Dispute resolution	As an admin, I resolve or reject account disputes and trigger auto-unblock.	HIGH	AdminDisputesPage, AccountDispute model
Platform settings	As an admin, I toggle platform-wide settings (listing fee required, etc.) without a deploy.	HIGH	PlatformSetting model, AdminSettingsPage
Notification settings	As an admin, I configure notification templates and rules.	MEDIUM	/api/admin/settings/notifications
Pricing configuration	As an admin, I change plan prices and credit counts without a deploy.	HIGH	/api/admin/pricing, 5-min cache via pricingConfigService
Telegram integration	As an admin, I configure the Telegram bot and share listings to a channel.	MEDIUM	telegramRoutes, AdminTelegramPage, requires configured token
Facebook integration	As an admin, I configure Facebook Graph API and share listings to a page.	MEDIUM	facebookRoutes, AdminFacebookPage, requires configured token
Premium request inbox	As an admin, I triage buyer	MEDIUM	AdminPremiumRequestsPage, PremiumRequest model

	premium-unlock requests.			

AI Assist

Description: OpenAI-powered assistants across two surfaces — a global AI chat widget for general product questions and the admin due diligence synthesizer that fuses FMCSA + CreditSafe data into a narrative report.

Narrative: AI is a productivity tool for admins first, users second. Due diligence synthesis is the highest-leverage use: one OpenAI call turns raw FMCSA safety data, authority history, insurance filings, and a CreditSafe report into a one-screen summary for the admin reviewer. Eva AI (consumer-facing) is a preview surface today, not a full assistant.

Requirement	User Story	Importance	Jira Issue	Notes
AI chat threads	As any user, I open an AI chat thread.	HIGH		POST /api/ai-chat/thread, OpenAI Assistants API
AI chat messaging	As any user, I send messages and receive responses in a thread.	HIGH		POST /api/ai-chat/message
Global AI chat widget	As any user, I access the AI widget from any page.	HIGH		AIChatWidget component, globally mounted
Admin due diligence synthesis	As an admin, I run an OpenAI-powered synthesis of FMCSA + CreditSafe data on any MC.	HIGH		GET /api/admin/due-diligence/analyze/:mcNumber, AdminAIDueDiligence
Eva AI preview	As a visitor, I preview the Eva AI assistant as marketing.	LOW		EvaAIPreviewPage — **preview only, not a production assistant**

Supporting Services

Description: Ancillary lead-generation flows and paid consultation booking that sit alongside the marketplace but are not part of the core buy-sell flow.

Narrative: These are conversion surfaces targeting the broader trucking operator audience — dispatch, safety, recruiting, fuel, and back-office services. Each form captures lead info and emails admin; there is no admin inbox page for submissions today. Paid 1:1 consultations run on their own Stripe flow (separate webhook endpoint).

Requirement	User Story	Importance	Jira Issue	Notes
Paid consultation booking	As a visitor, I book a paid consultation via Stripe Checkout.	MEDIUM		POST /api/consultations/checkout, Consultation model, own webhook at /api/consultations/webhook
Consultation admin console	As an admin, I manage, refund, and track paid consultations.	MEDIUM		AdminConsultationsPage
Dispatch services lead form	As a visitor, I submit a dispatch services inquiry.	LOW		POST /api/dispatch/submit, IP rate-limited, email-only
Safety services lead form	As a visitor, I submit a safety services inquiry.	LOW		POST /api/safety-services/submit
Recruiting services lead form	As a visitor, I submit a driver recruiting inquiry.	LOW		POST /api/recruiting-services/submit
Fuel program lead form	As a visitor, I submit a fuel program inquiry.	LOW		POST /api/fuel-program/submit
Back-office admin services lead form	As a visitor, I submit a back-office services inquiry.	LOW		POST /api/admin-services/submit

---

⚠️ Out of Scope

Not shipped today (implied or partial)

Review submission flow The `Review` model exists and `GET /api/users/:id/reviews` works, but there is no create-review endpoint or UI. Trust-score math that weights reviews cannot function today. Unblocking this would close a visible gap between the trust positioning and the actual implementation.

Saved listings Backend has `GET /api/listings/saved`, `POST /api/listings/:id/save`, `DELETE /api/listings/:id/save`. No frontend consumer exists. The `SavedListing` model is populated by nothing.

Seller verification and analytics pages `/api/seller/verification` and `/api/seller/analytics` return data the UI never renders. `SellerEarningsPage`, `SellerDocumentsPage`, and `SellerVerificationPage` are shells.

Admin analytics and broadcast `/api/admin/analytics/revenue`, `/users`, `/listings` and `POST /api/admin/broadcast` are all backend-only. Dashboard depth beyond the current KPI cards is not present. Placeholder routes exist (`/admin/reported`, `/admin/reports`) with only `<h1>` stubs.

Lead form admin inbox Dispatch, Safety, Recruiting, Fuel Program, and Admin Services forms email only. There is no admin page to browse, filter, or triage submissions.

Mobile app No React Native or Expo project. Socket.io patterns imply mobile-readiness but no mobile client exists.

2FA Not implemented. Called out in `IMPLEMENTATION_REPORT.md` medium-term list.

Subscription history `Subscription.userId` is `unique: true` — only one row per user. No plan-change history surface for users or admins.

Subscription renewal cron Renewal relies entirely on `invoice.paid` webhook events. No reconciler cron for missed webhooks.

Audit log viewer `AdminAction` rows are populated; only surfaced through the activity-log page with minimal filtering. No search or structured audit UI.

Admin impersonation Referenced in `IMPLEMENTATION_REPORT.md`; not implemented.

S3 production storage Storage service is stubbed. Documents currently write to local Heroku disk via Multer — ephemeral between deploys. Presigned URLs are not wired.

CI/CD, Sentry, API versioning All listed as short-term TODOs in `IMPLEMENTATION_REPORT.md`; none present.

Business intelligence & strategic analytics

This future capability would use platform data (listings, users, transactions, credits, deals by lane/state, seller performance, buyer cohorts) to produce operator-level insight.

What BI would eventually include (not in current scope):

Marketplace health metrics

- Listing velocity by state, MC age, fleet size, and trust tier
- Time-to-first-unlock and time-to-first-offer distributions
- Conversion funnel from marketplace entry → unlock → offer → transaction

Seller and buyer cohort analytics

- Seller earnings curves and payout latency
- Buyer subscription LTV by tier, churn reasons, reactivation rates
- Credit consumption patterns per plan

Financial intelligence

- Revenue by product (marketplace subs, CarrierPulse subs, credit packs, consultations, listing fees)
- Stripe dispute and refund trends
- Payout operations load (instant vs standard, time-to-release)

Trust signal effectiveness

- Conversion lift from CreditSafe report views
- False-positive rate on admin rejections
- Dispute volume vs trust tier

AI Business Analyst (Future Evolution)

A conversational admin assistant that answers:

- "Which listings have been sitting longest without an offer?"
- "Who are my top-earning sellers this quarter?"
- "Which buyer cohort churns fastest after a failed offer?"
- "What's my projected MRR at current new-sub velocity?"

🎨 Design

❓ Open Questions

Question	Answer	Date Answered
Seller offer actions (accept/reject/counter): should sellers act directly on forwarded offers, or does admin mediation continue to gate seller responses? Backend endpoints exist but no UI — product decision needed.		
Reviews: should buyer and seller be able to post reviews after a transaction closes? Trust score math assumes yes, but no flow exists.		
Saved listings: ship the UI or remove the backend endpoints?		
Listing fee enforcement: platform setting currently toggleable — do we commit to $49.99 enforced in production or keep opt-out?		
CarrierPulse + marketplace bundling: do we offer a combined plan, or keep them strictly separate Stripe subscriptions?		

🗂️ Reference Links

🛠️ Known Issues & Tech Debt

This section captures production issues and technical debt discovered during the reverse-engineering pass. Included here for engineering visibility; prioritize separately from new feature work.

Subscription sync bugs (load-bearing)

`stripeSubId` vs `stripeSubscriptionId` column name mismatch The Sequelize `Subscription` model column is `stripeSubId`. However, `creditService.ts` (lines 40, 388, 404, 458, 525, 530, 564, 903) and `buyerService.ts:467` write and query `stripeSubscriptionId`, which is not a column. Writes via the `/api/credits/subscribe` path silently drop the Stripe subscription ID. Webhook reconciliation via `where: {`

`stripeSubId: ... }` cannot find these rows. **Impact:** subscriptions created via `creditService.subscribe` are orphaned from their Stripe subscription.

`cancelledAt` vs `canceledAt` spelling mismatch Model column is `cancelledAt` (double-L, British spelling). `webhookController.ts:353, 378` writes `canceledAt` (single-L, American). Cancellation timestamps are silently dropped.

Webhook writes phantom columns `handleSubscriptionCreated/Updated/Deleted` in `webhookController.ts:320-323, 348-355, 376-379` calls `Subscription.upsert`/`update` with `currentPeriodStart`, `currentPeriodEnd`, `cancelAtPeriodEnd` — none of which exist on the model. Stripe period boundaries are not mirrored.

Status enum case mismatch Model `SubscriptionStatus` enum is UPPERCASE (`ACTIVE`, `CANCELLED`, `EXPIRED`, `PAST_DUE`). Webhook writes Stripe's lowercase strings directly (`active`, `canceled`, `past_due`). Queries filtering by uppercase status can miss webhook-written rows.

`Payment` column mismatches `handlePaymentIntentFailed` writes `Payment.failureReason` (doesn't exist on model). `handlePaymentIntentSucceeded` writes `Payment.paidAt` (model has `completedAt`). Silent data loss.

Dual credit ledger drift Unlocks decrement `users.usedCredits` only. `Subscription.creditsRemaining` is written at grant time and never decremented. The subscription column is effectively vestigial — do not trust it for reporting.

Broken endpoints

Token refresh URL mismatch Frontend `api.refreshToken` posts to `/auth/refresh`. Backend mounts `/auth/refresh-token`. Token refresh silently fails; users are forced to log in again when access tokens expire.

Ad-hoc migration scripts (repo root)

No Sequelize migration tooling is in use. Schema drifts have been patched by hand-rolled scripts in the repo root:

- `migrate-vip.js` — adds `listings.isVip` + indexes
- `migrate-professional-to-premium.js` — migrates PROFESSIONAL subscribers to PREMIUM, shrinks ENUM, renames platform settings
- `fix-vip-enum.js` — extends `subscriptions.plan` ENUM with `VIP_ACCESS`, then hard-codes one user (`yamil@morpro.io`) to VIP

- `set-vip-user.js` — force-upserts a VIP subscription with no Stripe backing, bypassing the three-layer sync rule

Each script hints at schema drift that had to be hot-patched against production. Standardize on a migration runner (umzug, sequelize-cli migrate, or Prisma migrate) before the next schema change.

Orphan and half-built code

- `MCDetailPage.tsx` (v1) is in the tree; router uses `MCDetailPageV2` everywhere. Delete or merge.
- `CreateListingPage.tsx` is orphaned; seller flow uses `SellerCreateListingPage`.
- `Message` model declares a `listing` belongsTo in the class but the association is not registered in the associations block. Queries with `include: [Listing]` on Message will fail.
- `checkListingUnlocked` in `api.ts:1154` fetches up to 1,000 unlocked listings just to check membership. N+1 anti-pattern — replace with a dedicated endpoint.
- `handleChargeDisputeCreated` in webhookController is stubbed with a TODO for admin notification.
- `handlePaymentIntentSucceeded` PaymentType.LISTING_FEE branch has a TODO for listing status update.

Stale documentation

- `CLAUDE.md` subscription plan table is wrong on all four plans and off by ~20× on VIP_ACCESS. Canonical source is `src/services/pricingConfigService.ts`.
- `src/types/index.ts:210-264` `SUBSCRIPTION_PLANS` constant is stale. Unused at runtime but misleading.
- `IMPLEMENTATION_REPORT.md` (Dec 2025) still lists unresolved model updates and known issues that have not been fixed.

Deprecated enums still live

- `SubscriptionPlan` enum still includes `PROFESSIONAL`. The `requireProfessionalSubscription` middleware still exists. Code and DB drifted after the PROFESSIONAL → PREMIUM migration.
- `PACKAGE_TOOL` is the Pulse Bundle subscription ($14.99). Valid, but name doesn't match product naming — consider renaming enum to `PULSE_BUNDLE`.

Seller offer UX gap (flagged)

Backend has `POST /offers/:id/accept`, `/reject`, `/counter`. Frontend has no call sites. `SellerOffersPage` displays offers but has no action buttons. Sellers cannot act on forwarded offers from the UI today — flagged as a product decision (bug vs feature) in Open Questions.

Rate limiter duplication

`dispatchRoutes`, `adminServicesRoutes`, `safetyServicesRoutes`, `recruitingServicesRoutes`, `fuelProgramRoutes` each re-declare `express-rate-limit` config instead of importing the central limiter factory in `rateLimiter.ts`.

Untyped admin responses

Many admin controller handlers return `any`-typed responses. `adminController.ts` exports are repeatedly `asyncHandler(…) => any`. Typed DTOs would help the frontend consume these reliably.

End of PRD.